



Arrays & Core Operations

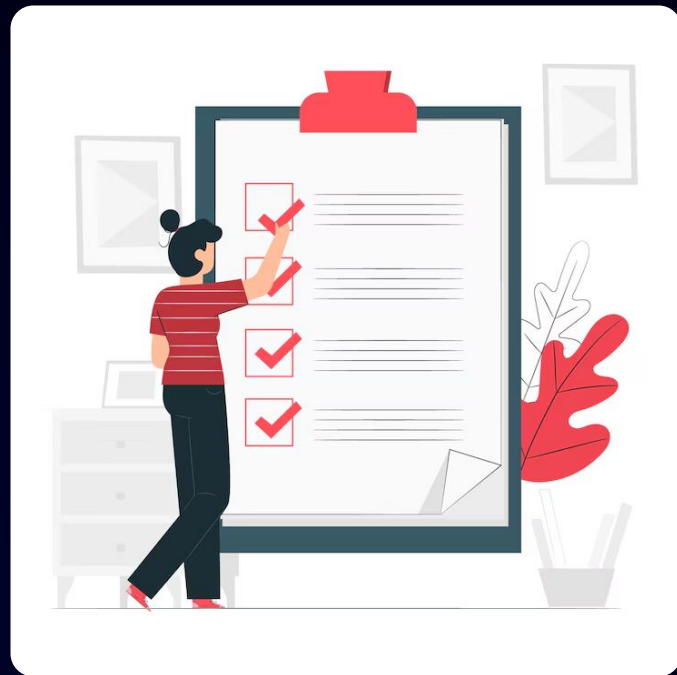
Learning Objectives

- Understand what arrays are and why they are used
- Create arrays with different types of values
- Access and modify array elements using index
- Iterate through arrays using loops
- Add and remove elements using array methods
- Differentiate between `slice()` and `splice()`
- Use useful array methods like `reverse()`, `join()`, `toString()`
- Understand why `delete` is not recommended for arrays



Prerequisites

- Basic understanding of variables
- Knowledge of let and const
- Basic JavaScript syntax
- Understanding of console.log()
- Basic loops (for loop) knowledge (helpful but optional)

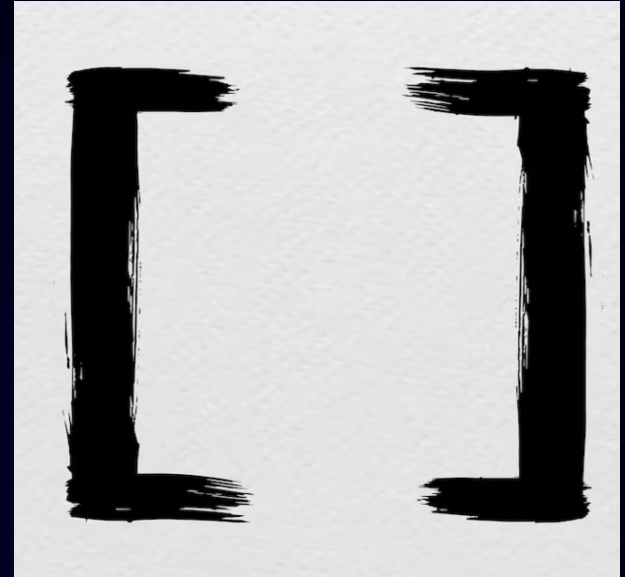




Introduction to Arrays

Arrays as Values

- An array stores multiple values in one variable
- Values inside an array are called elements
- Elements are written inside [] brackets
- Values are separated by commas
- Arrays keep related data together



creating an array

- Use `const` or `let` to declare an array variable
- Start with an array name followed by `=`
- Add values inside square brackets `[ ]`
- Arrays can also be created empty and filled later
- Elements are stored in order

Example : `let colors = ["Red", "Green", "Blue"];`

Creating an Array

- Use `const` or `let` to declare an array variable.
- Start with an array name followed by `=`.
- Add values inside square brackets `[ ]`.
- Arrays can also be created empty and filled later.
- Elements are stored in order.

Example

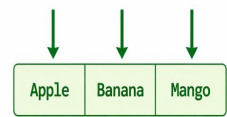
```
// Creating an array
const fruits = ["Apple", "Banana", "Mango"];

// Empty array
let numbers = [];

// Adding values later
numbers[0] = 10;
numbers[1] = 20;
```

`fruits = ["Apple", "Banana", "Mango"]`

0 1 2

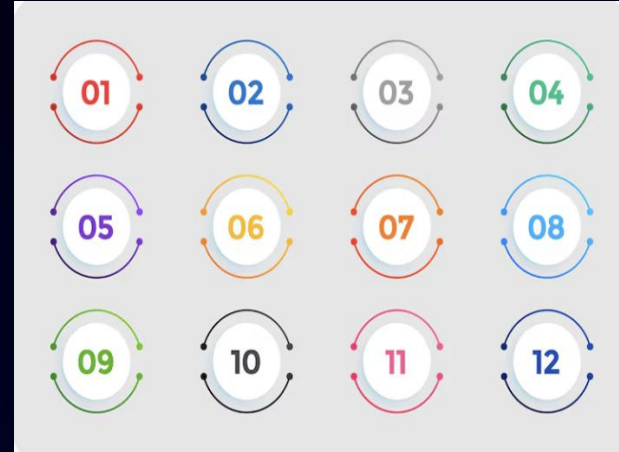


Indexes

Understanding Array Indexing

- Each element in an array has a position number
- This position number is called an index
- Indexing in arrays starts from 0
- First element is at index 0
- Second element is at index 1
- Index is used to access specific elements

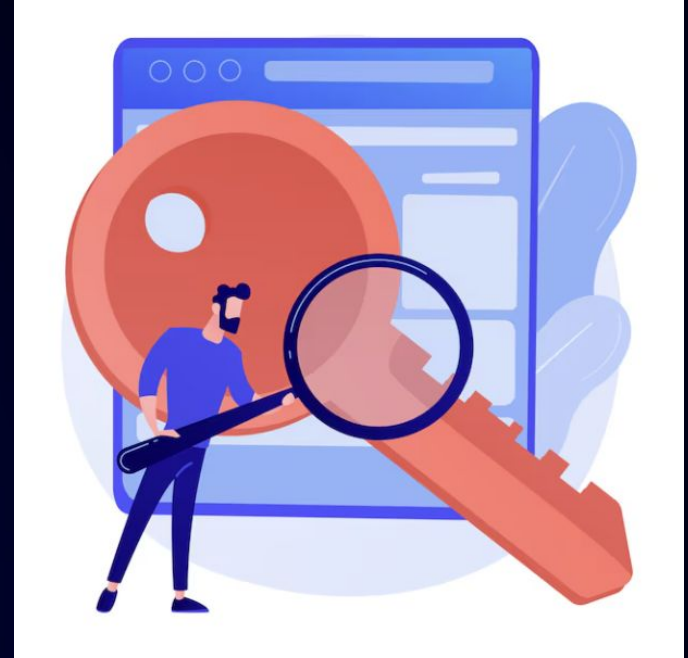
Example : `const numbers = [10, 20, 30];`
`console.log(numbers[0]); // 10`
`console.log(numbers[2]); // 30`



Accessing Array Elements

- Use index number to access array values
- Write array name followed by square brackets []
- Put the index inside the brackets
- `console.log()` is used to display the value
- Index 0 gives the first element
- You can access any element using its index

Example: `console.log(numbers[0] ;`



Updating Array Elements

- Array values can be changed using index
- Write array name followed by index []
- Use = to assign a new value
- The old value gets replaced
- Only the selected element is updated



Pop Quiz

How do you change second element?

A

```
numbers(1) = 50
```

B

```
numbers{1} = 50
```

Pop Quiz

How do you change second element?

A

```
numbers(1) = 50
```

B

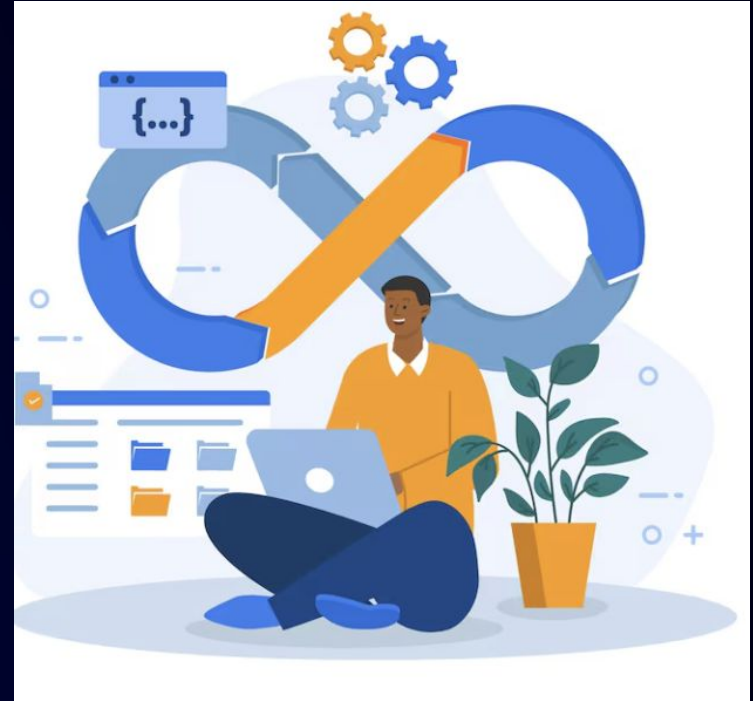
```
numbers{1} = 50
```



Looping Through Arrays with for...of

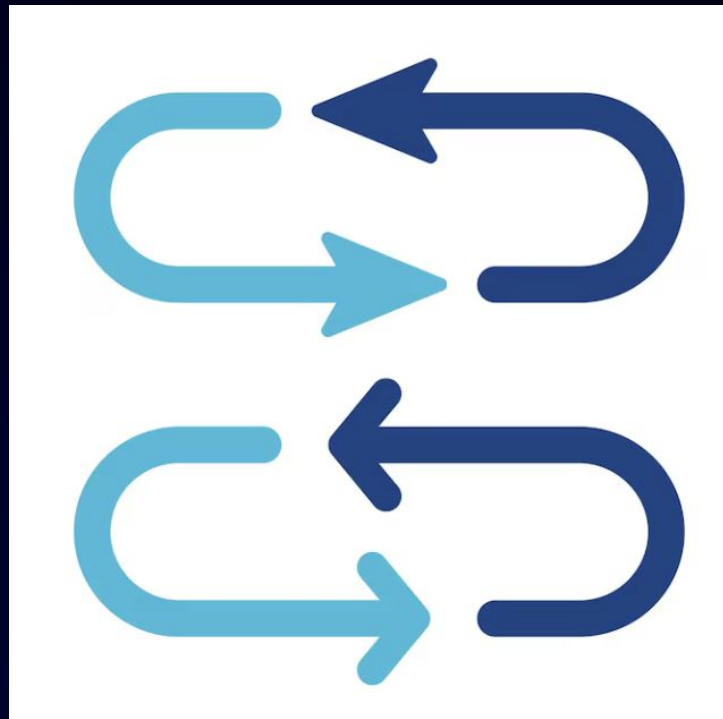
Why Iterate Through Arrays

- Arrays store multiple values that often need processing
- Iteration helps access each element one by one
- Useful for displaying all values in the array
- Helps perform the same operation on all elements
- Saves time compared to accessing each index manually
- Commonly used with loops like for and for...of



for...of vs Traditional for Loop

- for...of directly gives array values
- Traditional for loop uses index number
- for...of is shorter and easier to read
- Traditional loop gives more control using index
- Both are used to iterate through arrays



Pop Quiz

Which loop is used to iterate over array values directly?

A

while

B

for...of

Pop Quiz

Which loop is used to iterate over array values directly?

A

while

B

for...of



Array Methods: Add & Remove Elements

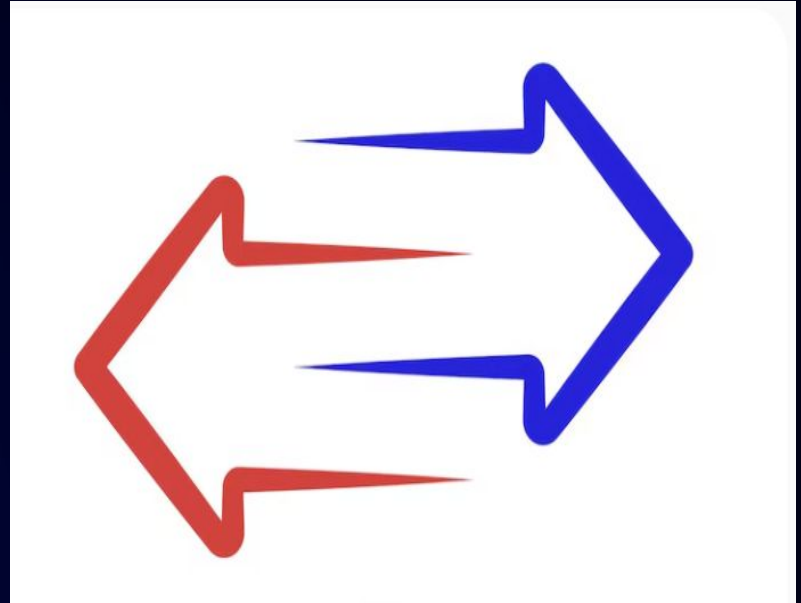
Adding & Removing Elements

- Arrays allow adding new elements
- Elements can be added at the beginning or end
- Arrays also allow removing elements
- Methods are used to modify array size
- Common methods: `push()`, `pop()`, `shift()`, `unshift()`



Array Methods:

- `push()` → adds an element to the end of array
- `pop()` → removes the last element
- `unshift()` → adds an element at the beginning
- `shift()` → removes the first element
- These methods change the original array
- Useful for dynamically updating arrays



Pop Quiz

Which method adds an element at the end of an array?

A

`pop()`

B

`push()`

Pop Quiz

Which method adds an element at the end of an array?

A

`pop()`

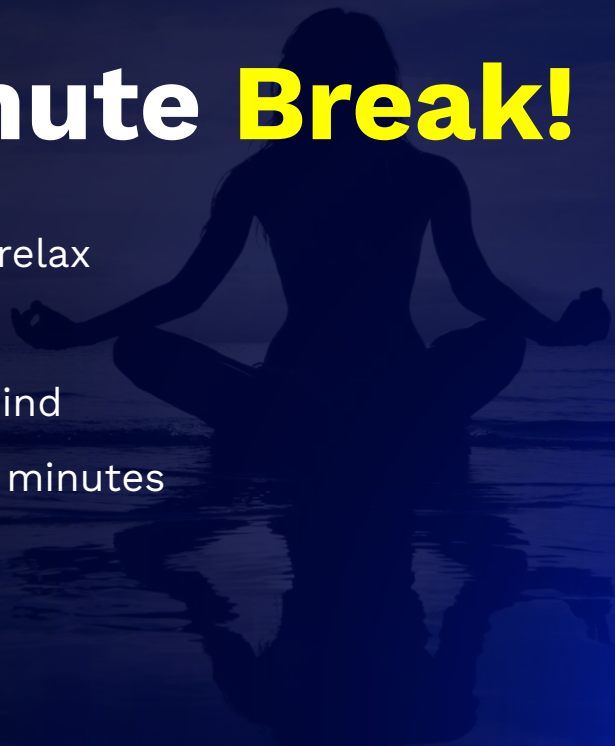
B

`push()`



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





slice vs splice

slice() vs splice()

- slice() → returns a portion of an array
- slice() does not change the original array
- splice() → adds, removes, or replaces elements
- splice() modifies the original array
- slice(start, end) → end index not included
- splice(start, deleteCount, items) → can insert/remove



Extract Part of an Array

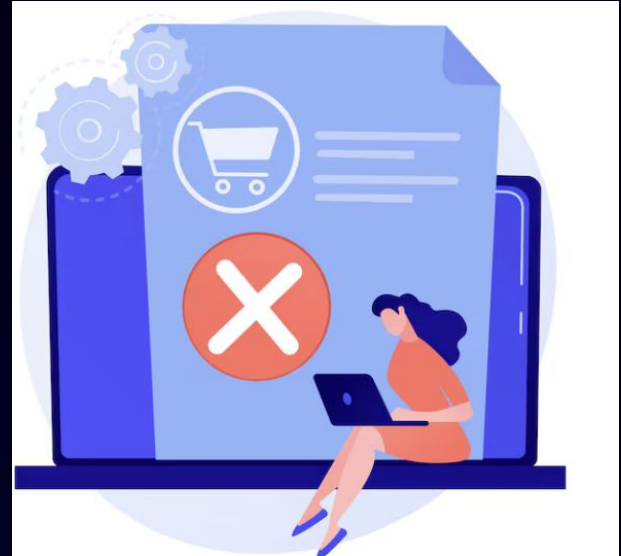
- `slice()` is used to extract part of an array
- It creates a new array with selected elements
- Original array remains unchanged
- Provide start and end index
- End index is not included

Example: `const numbers = [10, 20, 30, 40];`
`const result = numbers.slice(1, 3);`
`console.log(result); // [20, 30]`

Remove Elements from Array

- `splice()` is used to remove elements from an array
- It changes the original array
- Provide start index and number of elements to remove
- Elements are removed from the given position
- Can remove one or multiple elements

Example : `const numbers = [10, 20, 30, 40];`
`numbers.splice(1, 2);`
`console.log(numbers); // [10, 40]`



Pop Quiz

Which method extracts part of an array without changing original?

A

`splice()`

B

`slice()`

Which method extracts part of an array without changing original?

A

`splice()`

B

`slice()`



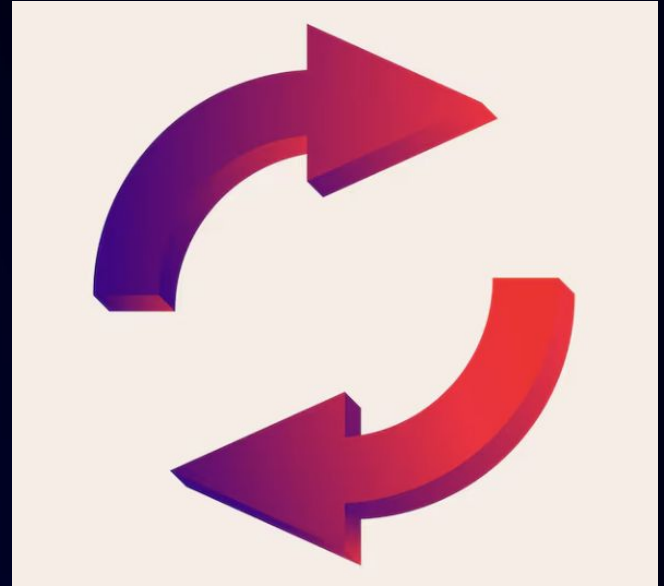
Helpful Array Methods

reverse() Method

- `reverse()` is used to reverse array order
- First element becomes last
- Last element becomes first
- It modifies the original array
- No parameters are required

Example:

```
const fruits = ["Apple",  
"Banana", "Mango"];  
console.log(fruits.join(" -  
"));
```



join() Method

- join() converts array into a string
- Elements are combined into one text
- You can provide a separator
- Comma is default separator
- Original array remains unchanged

Example :

```
const fruits = ["Apple", "Banana",  
"Mango"];  
console.log(fruits.join(" - "));
```



toString() Method

- **toString()** converts array to string
- Elements separated by commas
- No separator can be customized
- Original array is not modified
- Similar to join() with comma

Example:

```
const numbers = [10, 20, 30];  
console.log(numbers.toString());
```



Which method converts array to string with custom separator?

A

`reverse()`

B

`join()`

Pop Quiz

Which method converts array to string with custom separator?

A

`reverse()`

B

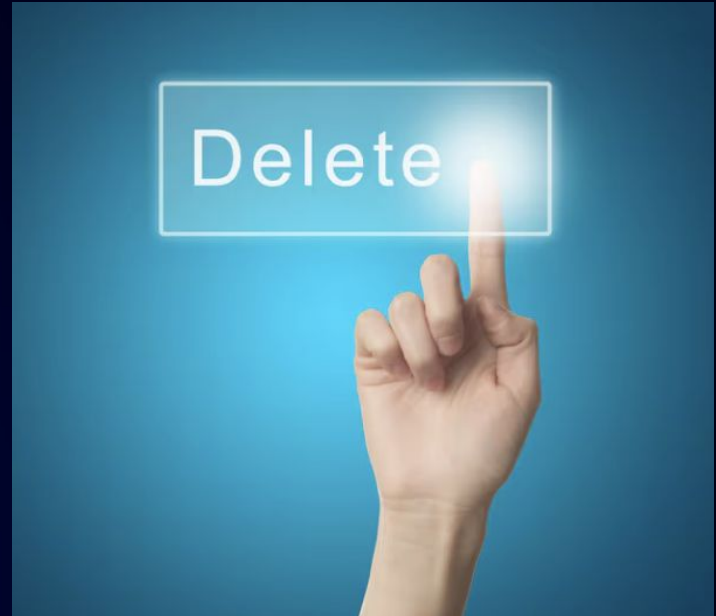
`join()`



Delete Operator

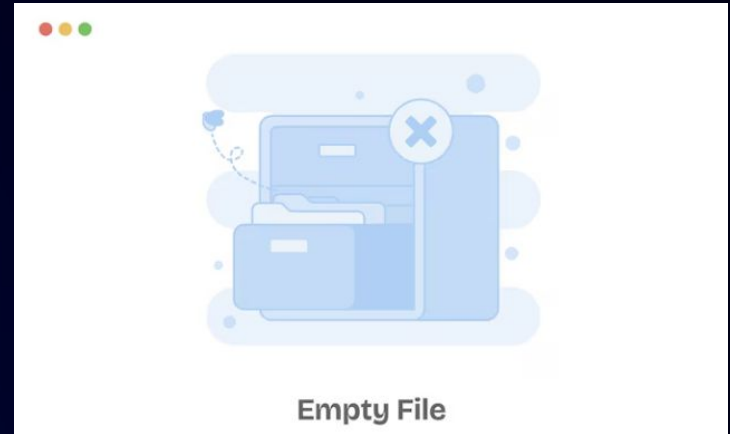
delete Operator in Arrays

- delete removes an element using index
- It does not change array length
- The position becomes empty (undefined)
- It leaves a gap in the array
- Not recommended for removing elements
- splice() is preferred for proper removal



delete Keeps Empty Slot

- delete removes only the value
- Index position remains in the array
- Creates an empty slot in that position
- Array length does not change
- Can lead to unexpected empty values
- splice() is better for proper removal



delete is Not Preferred for Removal

- delete removes only the value, not the index
- It leaves an empty slot in the array
- Array length remains the same
- Can cause issues while looping
- Does not shift elements automatically
- Use splice() for proper removal



What happens to array length after using delete?

A

Decreases

B

Remains same

What happens to array length after using delete?

A

Decreases

B

Remains same



Hands on Practice

create array of names and print all

- **Print all**
- **add/remove elements using push/pop**
- **reverse**
- **extract subset using slice**
- **remove elements using splice**

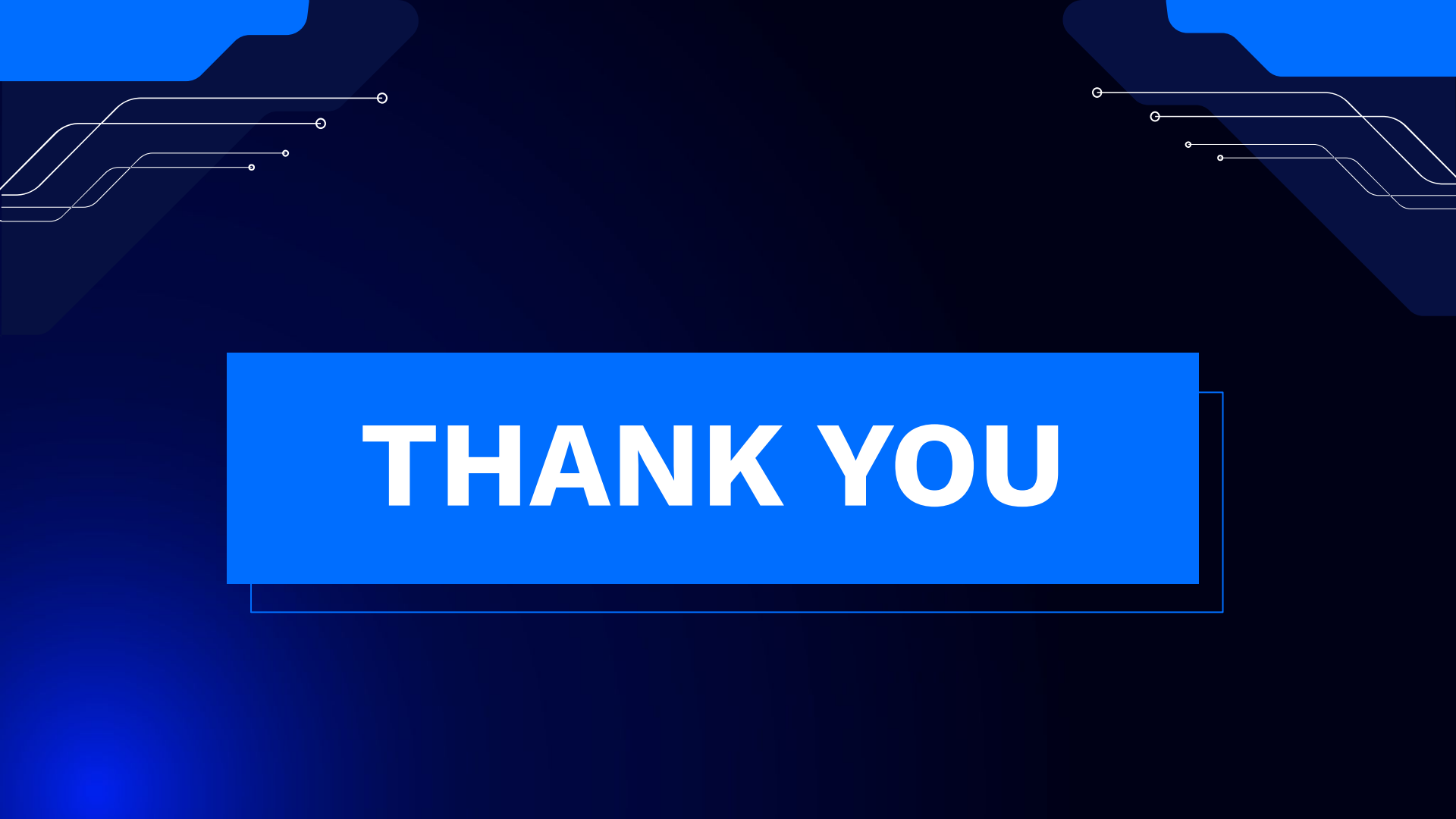
Summary

- Arrays store multiple values in one variable
- Index starts from 0 to access elements
- Arrays can be created, accessed, and modified
- Use loops to iterate through array values
- Add/remove elements using `push()`, `pop()`, `shift()`, `unshift()`
- `slice()` extracts and `splice()` modifies arrays
- `reverse()`, `join()`, `toString()` are useful methods
- `delete` creates empty slot (not preferred)





DOUBT RESOLUTION



THANK YOU